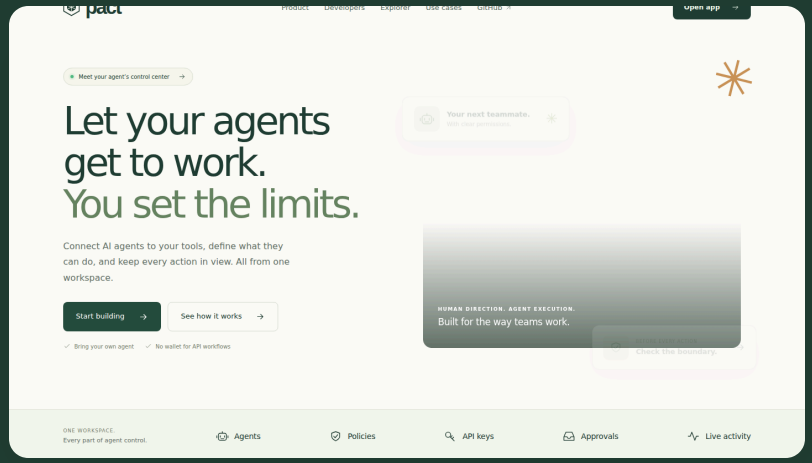
Pact

Programmable authority for autonomous agents

Connect an agent. Set its boundaries. See every decision. Verify an economic receipt on Monad when the action settles.



Agents now act in real systems.

Access is easy. Accountability is harder.

A wallet or provider key grants broad power. A prompt does not establish who authorized an action, what liability it creates, or why a service accepted it.

One enforceable boundary.

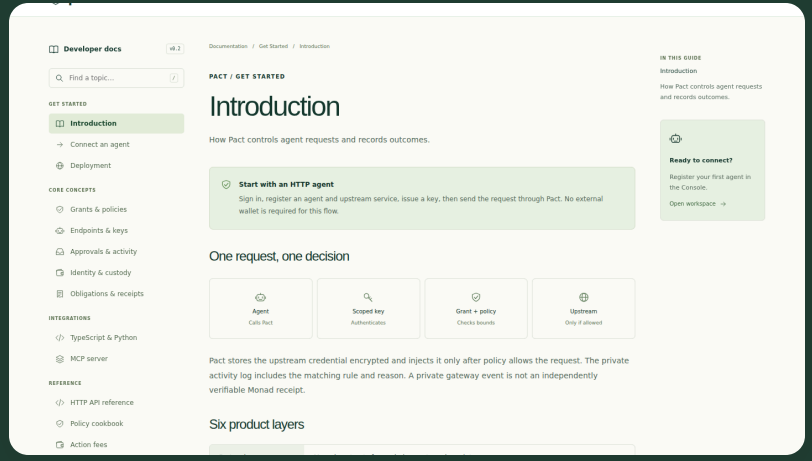
Identity → authority → decision → outcome

An owner grant defines authority. Ordered policies check each proposed action. A scoped Pact key authenticates the integration. The gateway forwards only allowed calls and records the decision.

Developer-first setup.

Agent → endpoint → secret → grant → policy → key

Register operations manually or import OpenAPI. Protect upstream credentials in the vault. Give the agent a scoped Pact key, never the upstream secret. TypeScript, Python, curl and MCP clients use the same API.



The gateway checks first.

Allowed GET forwards. Forbidden method stops.

The gateway authenticates the key, checks agent and endpoint state, evaluates ordered rules and grant limits, then injects the protected credential only for an allowed upstream call. Denials do not reach the upstream.

Every decision has a trace.

Agent · endpoint · rule · reason · request ID

Live Activity shows allowed, denied and pending decisions with upstream status and latency. HTTP activity is a private operator record, retained under product policy; it is not itself an onchain receipt.

Reserve the full exposure.

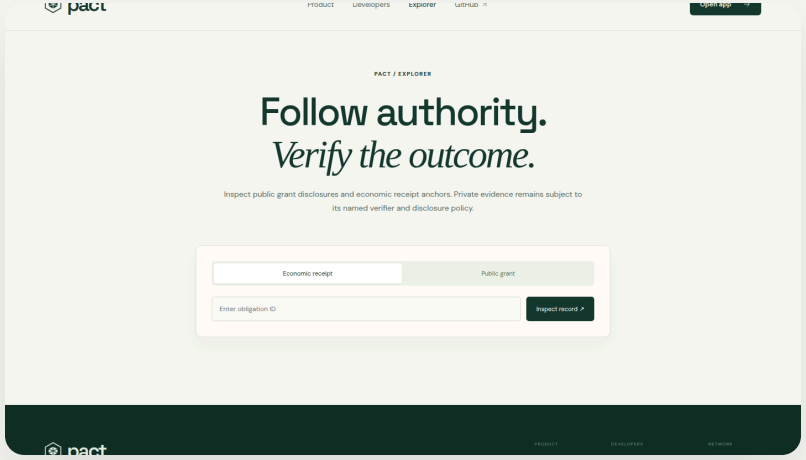
A bound is on the commitment, not only the first charge.

Monad testnet grants constrain agent, token, recipient, per-action amount and aggregate capacity. Contracts reserve exact action capacity, prevent replay and require owner approval above threshold. Recurring liability valuation remains future work.

From promise to settlement.

Terms → escrow → evidence → verdict → receipt

The shipped path supports one funded milestone. Pact links signed terms, evidence, a named verifier verdict, settlement and a receipt root that can be checked against Monad testnet.



A real testnet exercise.

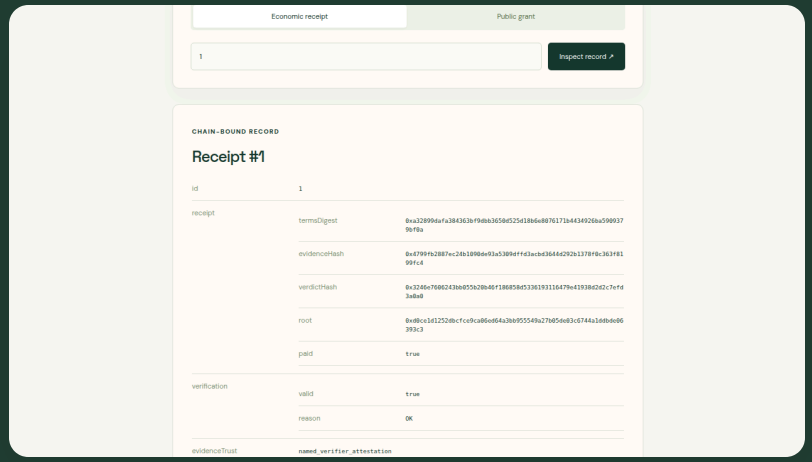
51 scenarios + 2 operational checks recorded

A separately deployed Railway agent used a real model through Pact, made allowed and denied gateway calls, executed approved test-token transfers and settled a funded obligation. The test actor later expired and its grants were revoked.

Inspect the receipt.

Monad testnet · public receipt 1

The public API reports verification.valid = true. The receipt root matches the recorded chain state. This does not prove real-world delivery or independent assessment.



Built for agent developers.

Start with one operation and one rule.

First users: teams protecting third-party API calls and Monad-based service or treasury agents. Open-source docs, SDKs, MCP tools and a local demo upstream make the integration reproducible. No customer or revenue claim is made.

Try Pact.

Live product · source · verifiable testnet evidence

https://pact-protocol-monad.alemzdelight.workers.dev/
https://github.com/linoxbt/Pact
https://pact-api-production-2ab9.up.railway.app/v1/receipts/1